Software Engineering

Lecture 4

What is a Class Diagram?

A Class Diagram is a type of UML (Unified Modeling Language) diagram that shows the structure of a system by representing:

Classes

Attributes (variables)

Methods (functions)

Relationships between classes

It is used in object-oriented programming (OOP) to design software before coding.

Main Parts of a Class Diagram

Each class is drawn as a rectangle with 3 sections:

```
+-----------------+
| Class Name      |
+-----------------+
| Attributes      |
+-----------------+
| Methods         |
+-----------------+
```

1.Class Name

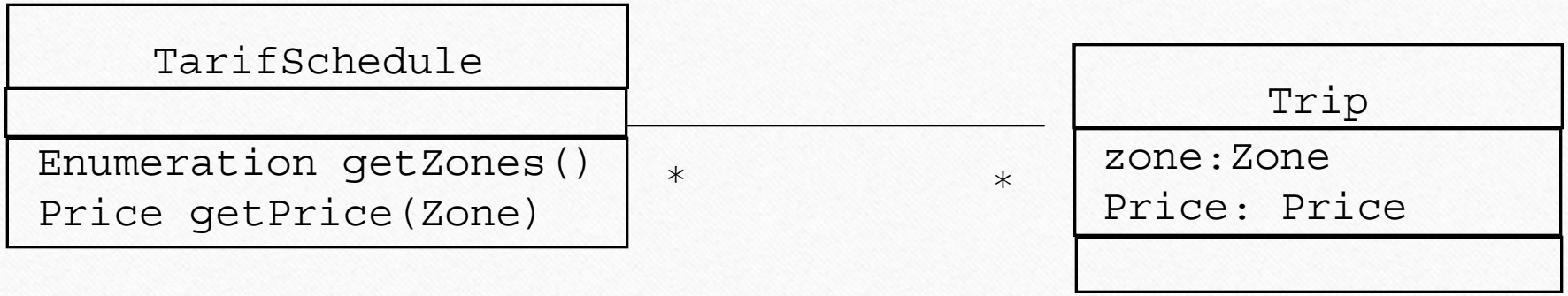
The name of the class

2. Attributes

Variables that store data

3. Methods

Functions the class can perform

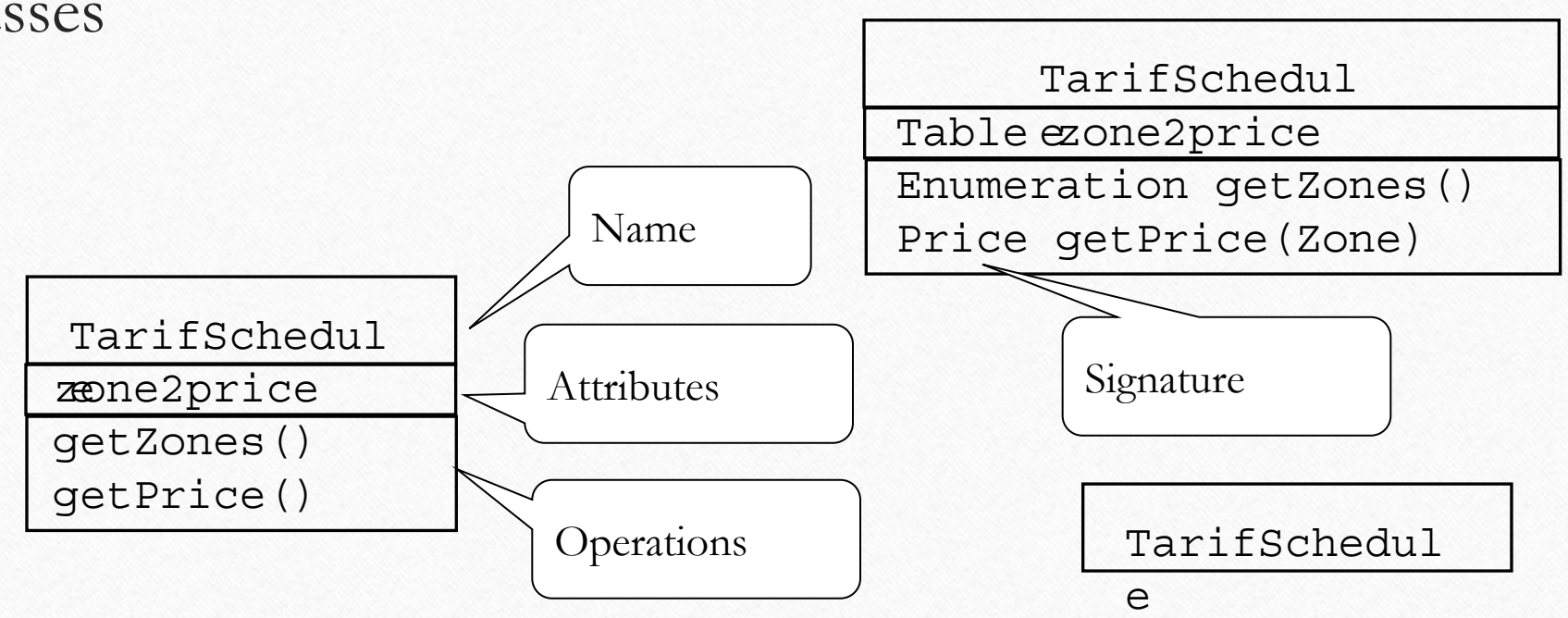


- Class diagrams represent the structure of the system.
- Used
  - during requirements analysis to model problem domain concepts
  - during system design to model subsystems and interfaces
  - during object design to model classes.

Actor vs Instances

- What is the difference between an *actor* , a *class* and an *instance*?
- Actor:
  - An entity outside the system to be modeled, interacting with the system (“Passenger”)
- Class:
  - An abstraction modeling an entity in the problem domain, must be modeled inside the system (“User”)
- Object:
  - A specific instance of a class (“Joe, the passenger who is purchasing a ticket from the ticket distributor”).

Classes

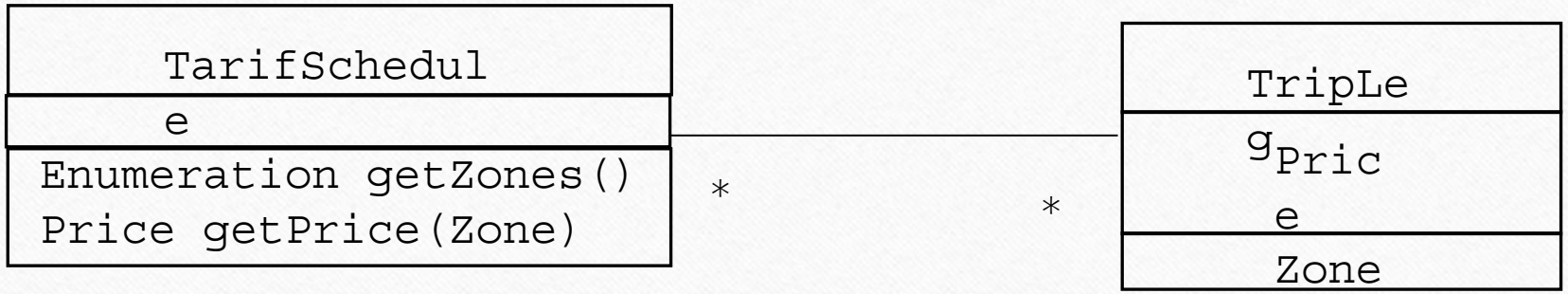


- A ***class*** represent a concept
- A class encapsulates state ***(attributes)*** and behavior ***(operations).***
- Each attribute has a ***type***.
- Each operation has a ***signature***.
- The class name is the only mandatory information.

Why Class Diagram is Important

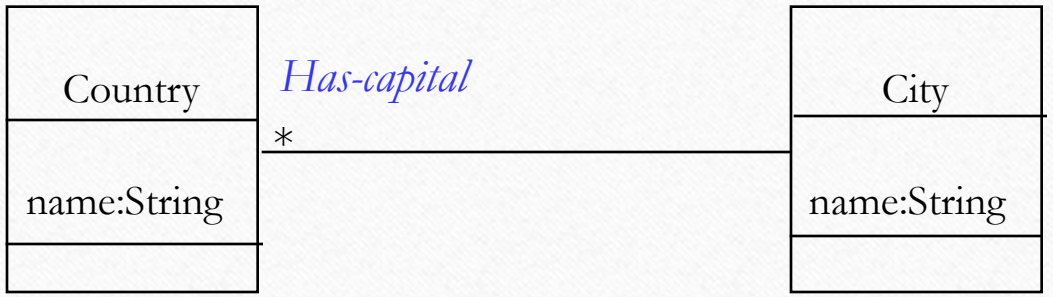
1. Helps plan software structure
2. Makes coding easier
3. Shows relationships clearly
4. Used in software engineering design and OOP

Associations

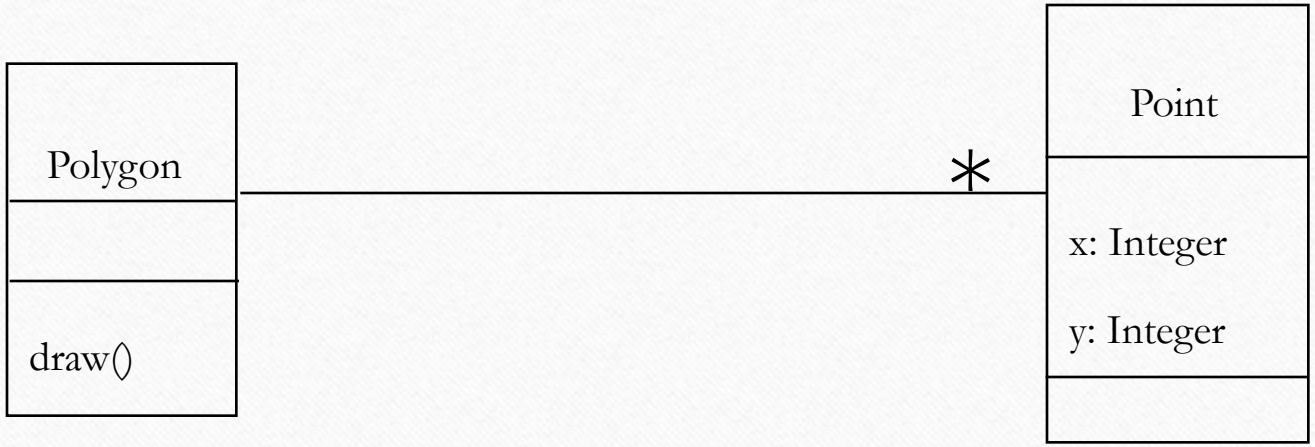


- Associations denote relationships between classes.
- The multiplicity of an association end denotes how many objects the source object can legitimately reference.

1-to-1 and 1-to-many Associations



One-to-one association

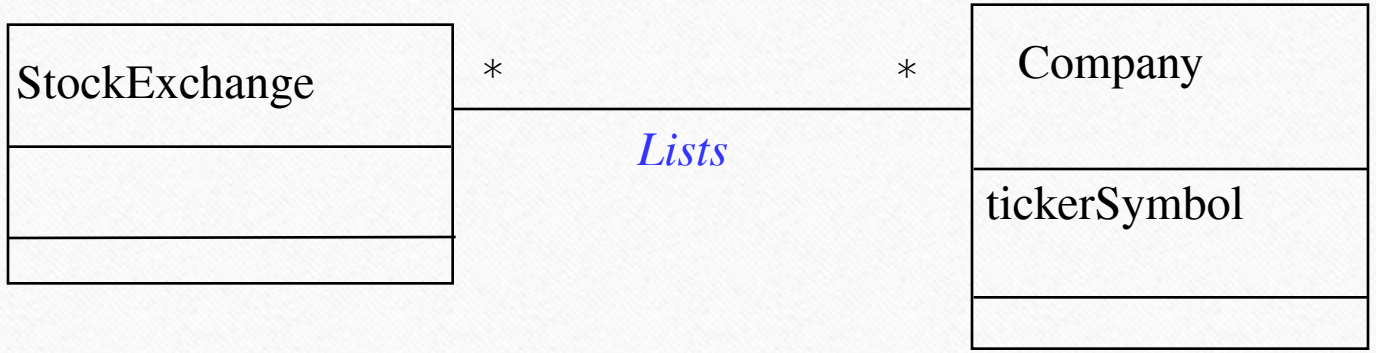


One-to-many association

From Problem Statement To Object Model

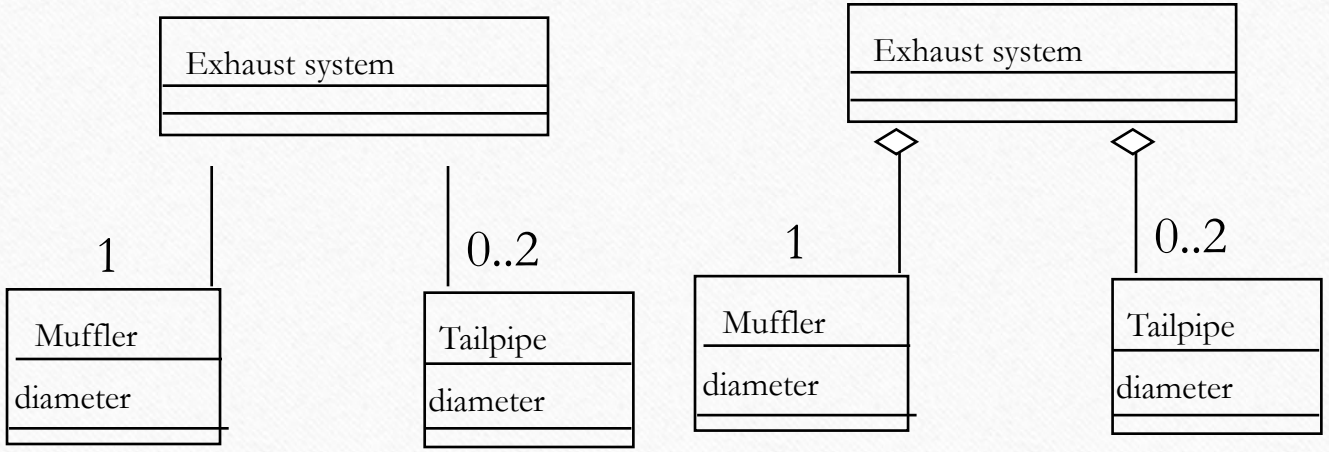
Problem Statement: A stock exchange lists many companies. Each company is uniquely identified by a ticker symbol

Class Diagram:

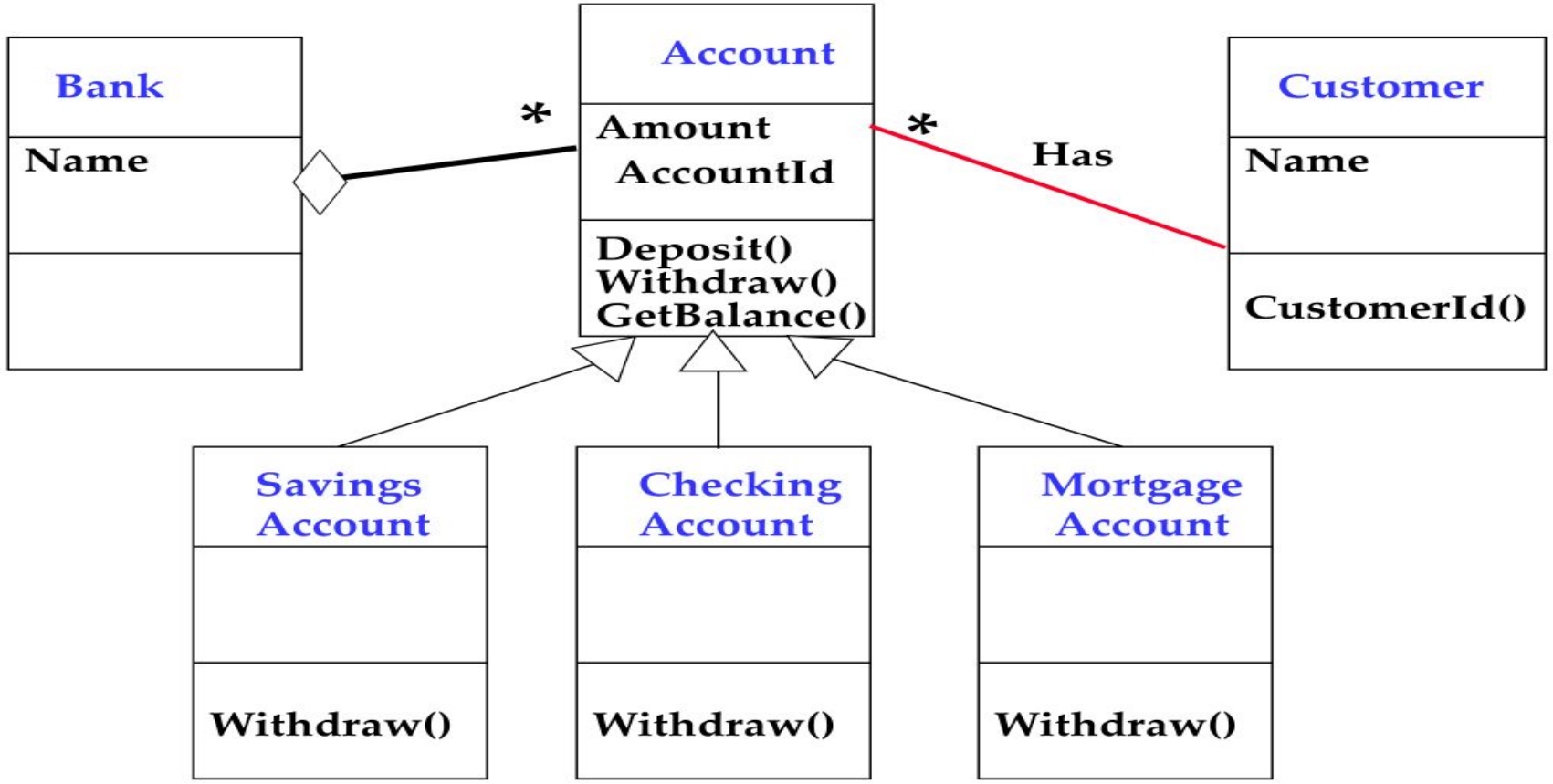


Aggregation

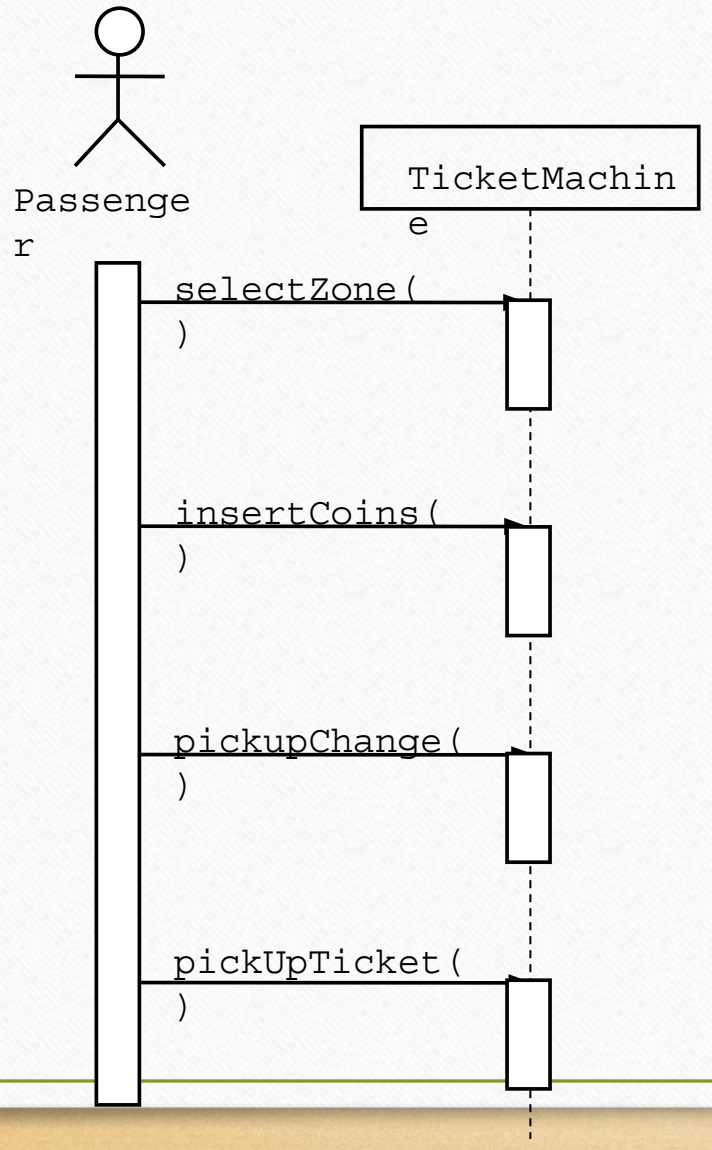
- An ***aggregation*** is a special case of association denoting a "consists of" hierarchy.
- The ***aggregate*** is the parent class, the ***components*** are the children class.



Practice Object Modeling: Iterate, Categorize!

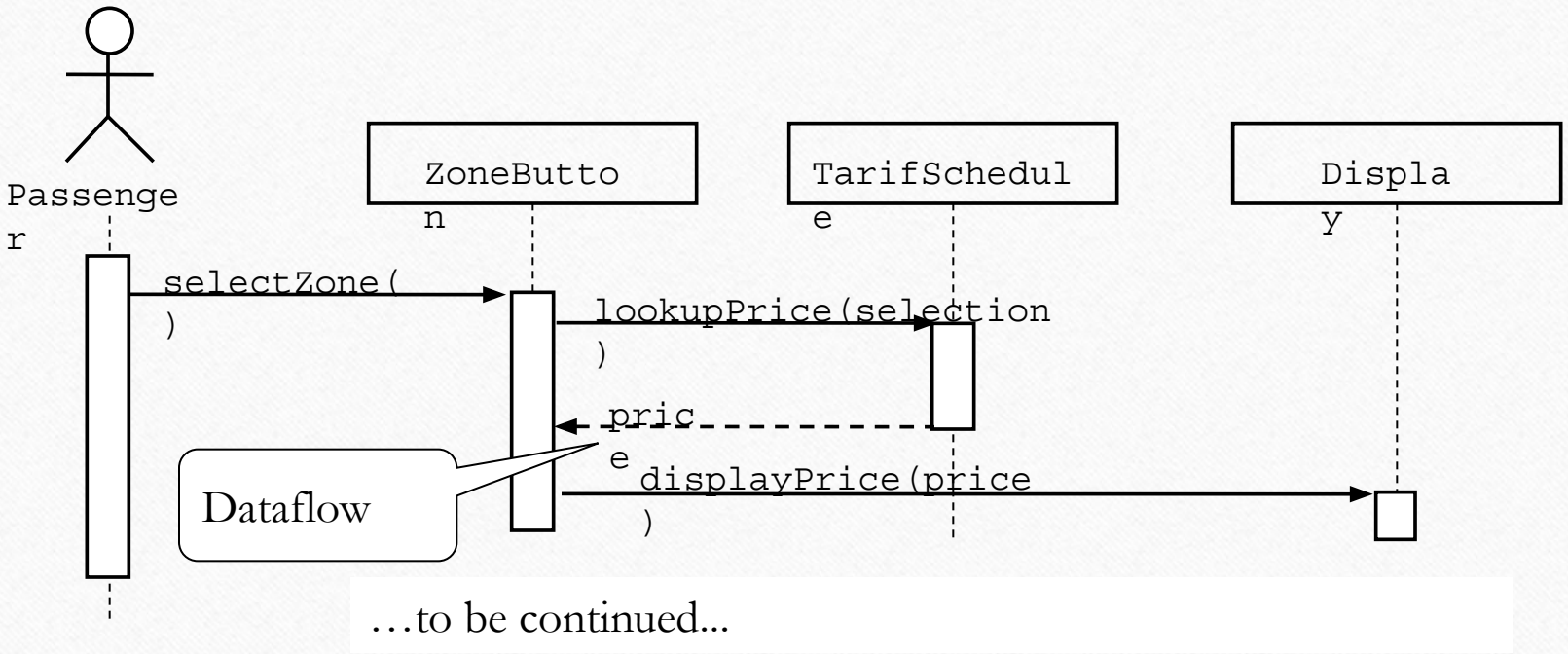


UML sequence diagrams



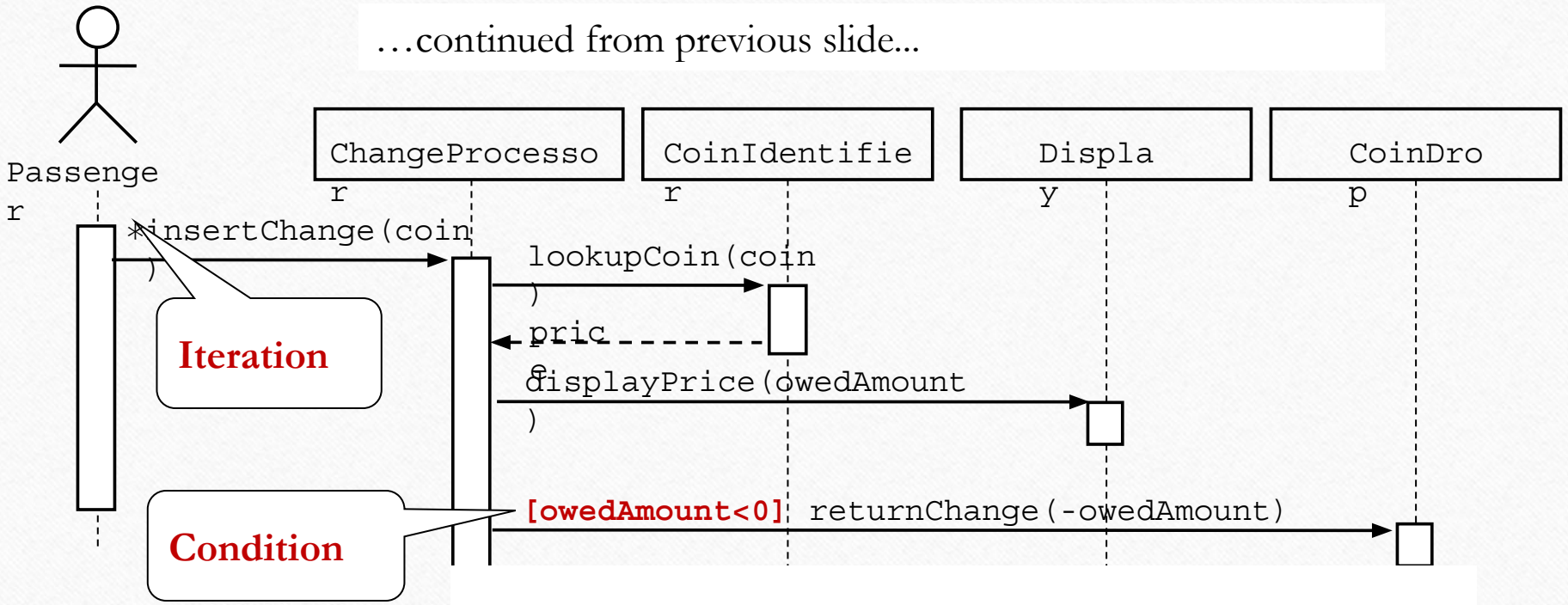
- Used during requirements analysis
  - To refine use case descriptions
  - to find additional objects ("participating objects")
- Used during system design
  - to refine subsystem interfaces
- ***Classes*** are represented by columns
- ***Messages*** are represented by arrows
- ***Activations*** are represented by narrow rectangles
- ***Lifelines*** are represented by dashed lines

Nested messages



- The source of an arrow indicates the activation which sent the message
- An activation is as long as all nested activations
- Horizontal dashed arrows indicate data flow
- Vertical dashed lines indicate lifelines

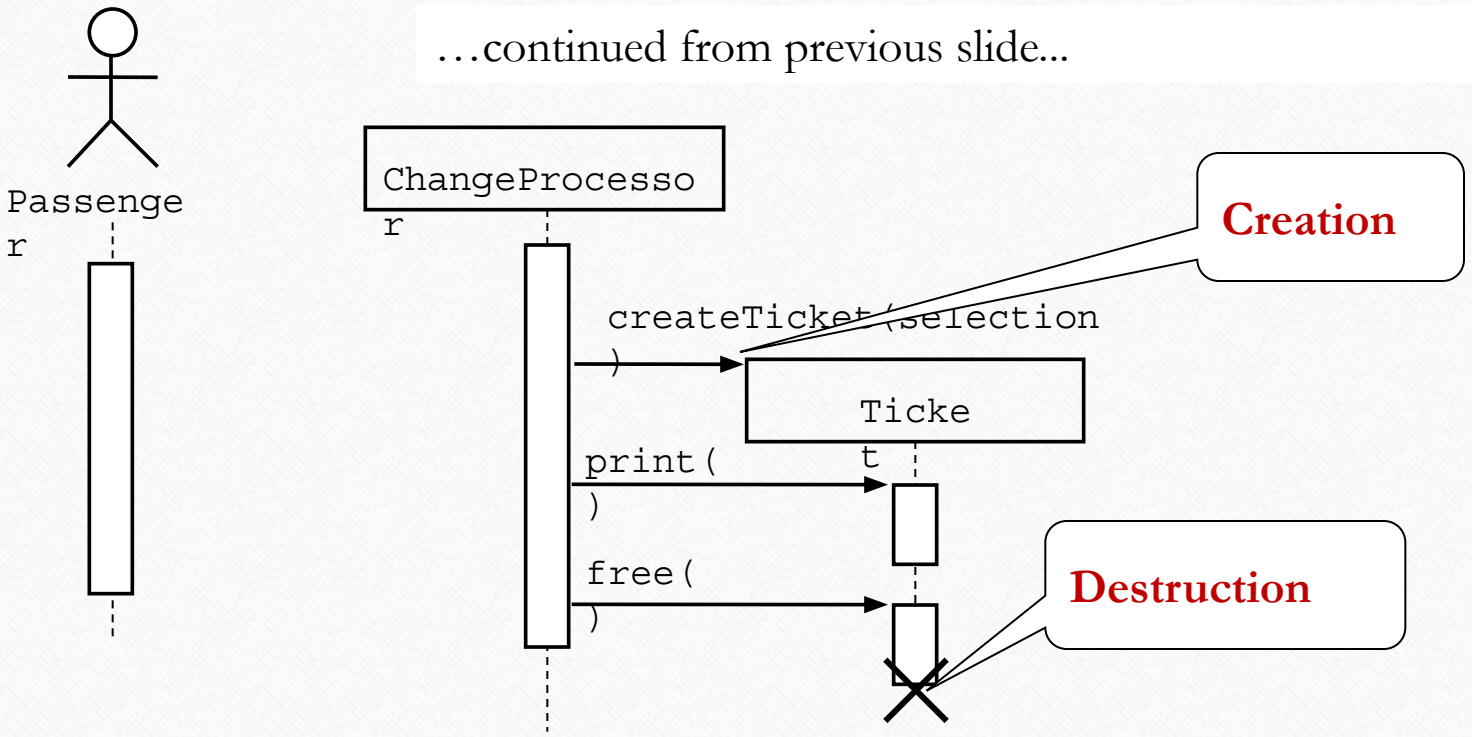
Iteration & condition



- Iteration is denoted by a * preceding the message name
- Condition is denoted by boolean expression in [ ] before the message name

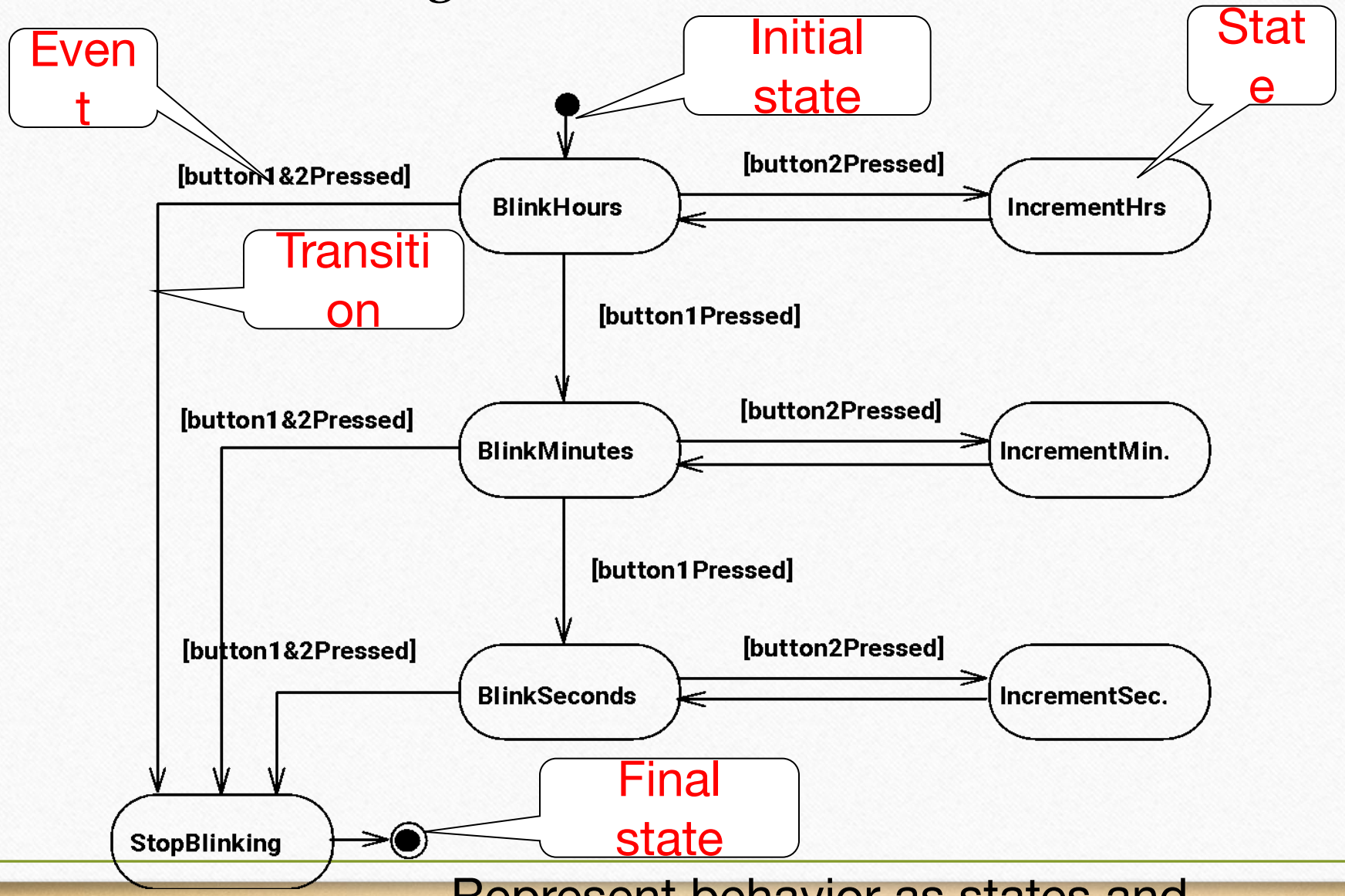
Creation and destruction

…continued from previous slide...



- Creation is denoted by a message arrow pointing to the object.
- Destruction is denoted by an X mark at the end of the destruction activation.
- In garbage collection environments, destruction can be used to denote the end of the useful life of an object.

State Chart Diagrams



Represent behavior as states and transitions